



Corso di Laurea Magistrale in Informatica

Programmazione Sicura 2025-2026

# Vulnerabilità di **Contact Discovery** in Apple AirDrop

---

## Studenti

Costantino Paciello

Vittorio Guida

## Prof.ssa

Barbara Masucci



# INDICE

- 01 Introduzione al Problema**  
Contesto, motivazione e obiettivi
- 02 Il Protocollo AirDrop**  
Discovery, autenticazione, data transfer
- 03 Il Fallimento Crittografico**  
SHA-256, bassa entropia, assenza di salt
- 04 Modello di Minaccia**  
Albero di attacco e threat modeling
- 05 Le Tre Vulnerabilità**  
Sender, Receiver e Log File Leakage
- 06 AirCollect**  
Proof of concept
- 07 Verifica Sperimentale della Vulnerabilità**  
Esperimento 1, 2 e 3
- 08 PrivateDrop**  
soluzione PSI
- 09 Risposta di Apple e Implicazioni**  
Analisi critica e Implicazioni

# 01 Introduzione al Problema

## 📶 Contesto

AirDrop è integrato in **1,5 miliardi di dispositivi** iOS/macOS. Opera offline tramite AWDL + BLE, senza server intermedi. Utilizzato quotidianamente in ambienti pubblici: metropolitane, aeroporti, università.

## 🛡️ Il Problema

Vulnerabilità di **Information Disclosure** scoperta nel 2021 (USENIX Security, ACM WiSec). Apple informata nel **maggio 2019**, ma al momento della pubblicazione nessuna patch era stata rilasciata.

## 🔧 Obiettivi del Progetto

Applicare la metodologia di Programmazione Sicura: individuazione debolezze → costruzione albero di attacco → analisi conseguenze → definizione mitigazioni.



L'attacco sfrutta la natura P2P e la mancanza di protezione crittografica adeguata sugli identificatori

# 02 Il Protocollo AirDrop

---

## 1 Discovery

Il mittente emette BLE con hash troncati (20 B). Il destinatario confronta con la rubrica → match → attiva AWDL.



## 2 Authentication

S invia HTTPS POST /Discover col Validation Record. R verifica firma Apple, UUID TLS e SHA-256 della rubrica.



## 3 Data Transfer

S invia Ask (metadati + anteprima), poi Upload (file completo). Connessione autenticata solo se entrambi hanno match in rubrica.



# 02 Il Protocollo AirDrop

## 🔑 Il Validation Record — Cuore della Vulnerabilità

Struttura firmata da Apple contenente:

- UUID account (cert.  $\sigma_{\text{UUID}}$ )
- SHA-256 di tutti gli ID:  $\text{SHA-256}(\text{ID}_1), \dots, \text{SHA-256}(\text{ID}_n)$
- Firma digitale Apple

Verifica dell'autenticità:

1. Verifica firma VR
2. Controllo UUID certificato  $\text{TLS} \leftrightarrow \text{VR}$
3. Confronto hash rubrica normalizzata

Visibilità: "Ricezione non attiva" / "Solo Contatti" (default) / "Tutti"

Le vulnerabilità interessano entrambe le ultime due modalità





# 03 Il Fallimento Crittografico

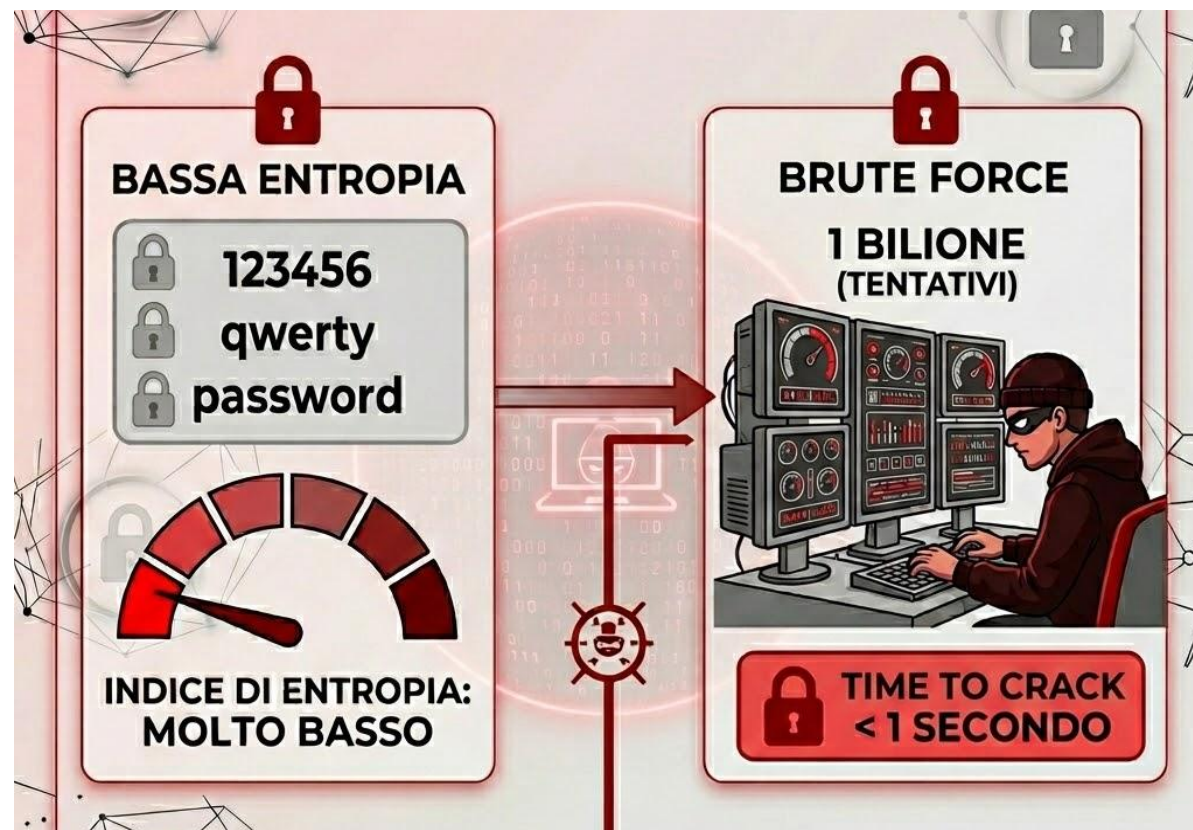
## 1. Bassa Entropia degli Identificatori

SHA-256 è sicuro, ma la sua resistenza dipende dall'entropia dell'input. Numeri di telefono: struttura rigida E.164, max 15 cifre.

$|N| = 10^{(l-p)} \rightarrow$  Cellulari tedeschi:  $10^7$  combinazioni = **23 bit**  
Tutti i numeri mondiali:  $\sim 10^{10} =$  **33 bit** (vs 128 bit password sicura)

### Entropia Comparata

|          |             |         |
|----------|-------------|---------|
| Password | <div></div> | 128 bit |
| Mondiale | <div></div> | 33 bit  |
| Germania | <div></div> | 23 bit  |



# 03 Il Fallimento Crittografico

## ✖ 2. Assenza di Salt

Apple non usa salt. Gli hash sono **statici** :  
stesso input = stesso output sempre.

$$h_i = \text{SHA-256}(\text{normalize}(\text{ID}_i) \parallel s)$$
  
Formula sicura (non usata)

$$h_i = \text{SHA-256}(\text{normalize}(\text{ID}_i))$$
  
Formula usata (insicura)



# 03 Il Fallimento Crittografico

## ⚡ 3. Rainbow Table Attack

Precomputazione offline: **76,5 MB** di tabelle coprono il **99,8%** dei numeri tedeschi. Generazione: pochi minuti su laptop consumer. Inversione: **1,2 ms** per hash.

### Mapping CWE

#### CWE-916

Use of Password Hash With Insufficient Computational Effort

#### CWE-760

Use of a One-Way Hash with a Predictable Salt (assente)

#### CWE-330

Use of Insufficiently Random Values





# 04 Modello di Minaccia e Albero di Attacco

## 🎯 Obiettivo dell'Attaccante

Information Disclosure: ottenere numero di telefono o email di un utente AirDrop **senza autorizzazione**, intercettando hash trasmessi dal protocollo.

## Le Tre Condizioni

- 1 Debolezza:** hash statici senza salt su identificatori a bassa entropia
- 2 Accessibilità:** vicinanza fisica radio o accesso a log di sistema
- 3 Capacità:** rainbow table precomutate, inversione in millisecondi

Se anche UNA condizione manca, l'attacco collassa



# 04 Modello di Minaccia e Albero di Attacco

Nodo Radice: Ottenere Identità Utente

OTTENERE NUMERO DI TELEFONO / EMAIL NON AUTORIZZATO

OR

Sender Leakage

Vittima apre pannello →  
trasmette hash BLE →  
attaccante passivo intercetta

Fattibilità: ALTA

Costo: BASSO

Receiver Leakage

Attaccante spoofa contatto  
noto → vittima risponde  
automaticamente → rivela hash

Fattibilità: ALTA

Costo: MEDIA

Log File Leakage

Accesso a log sysdiagnose →  
hash troncati (40 bit) →  
reidentificazione offline

Fattibilità: MEDIA

Costo: BASSO

Debolezza comune a tutti i rami: hash statici senza salt su identificatori a bassa entropia

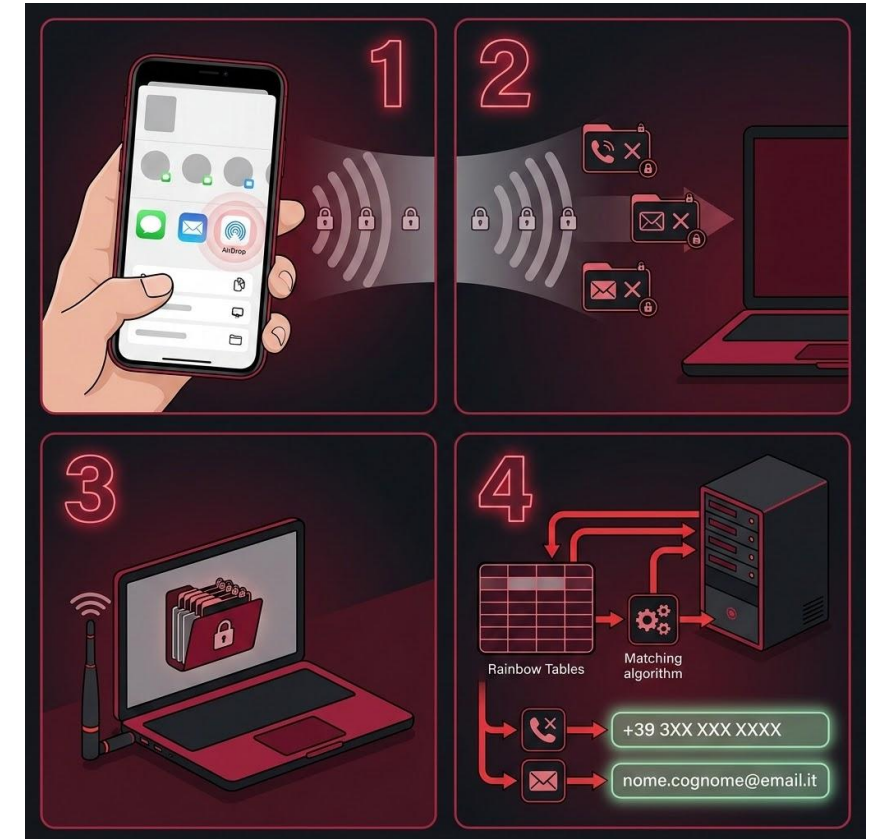
# 05 Le Tre Vulnerabilità

## “1” 1. Sender Leakage

### Meccanismo:

1. Vittima apre pannello condivisione
2. Dispositivo trasmette hash BLE in chiaro
3. Attaccante passivo intercetta (raggio 10-30m)
4. Inversione istantanea via rainbow table

**Impatto:** nessuna interazione richiesta. Funziona in modalità "Solo Contatti". Un laptop può monitorare decine di dispositivi simultaneamente.



Vicinanza Fisica

Sì

Interazione Vittima

No

Accesso Dispositivo

No

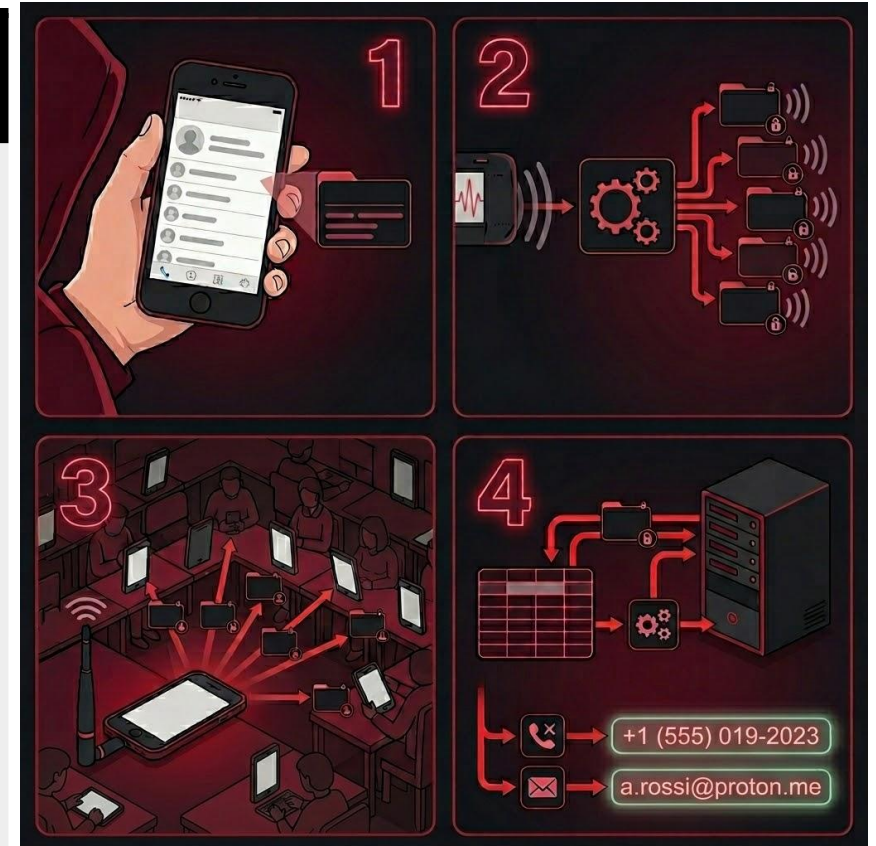
# 05 Le Tre Vulnerabilità

## ← 2. Receiver Leakage

### Meccanismo:

1. Attaccante individua contatto noto (es. centralino)
2. Calcola hash e costruisce pacchetti spoofati
3. Destinatari con quel contatto rispondono **automaticamente**
4. Attaccante raccoglie VR e inverte gli hash

Scenario pratico: hall aziendale, pausa pranzo. In pochi secondi: numeri personali di decine di dipendenti. Non richiede che la vittima apra AirDrop.



Vicinanza Fisica

Sì

Interazione Vittima

No (Auto)

Accesso Dispositivo

No



# 05 Le Tre Vulnerabilità

## 3. Log File Leakage

### Meccanismo:

1. Accesso fisico al dispositivo o autorità esegue sysdiagnose
2. Estrazione log AirDrop con hash troncati (40 bit)
3. Ricerca esaustiva: 40 bit sufficienti per reidentificazione
4. Identificazione retroattiva, anche a distanza di tempo

**Caso reale (gennaio 2024):** autorità cinesi hanno identificato manifestanti nelle metropolitane di Pechino e Shanghai analizzando log sysdiagnose.



Vicinanza Fisica

No

Interazione Vittima

No

Accesso Dispositivo

Si



# 06 AirCollect: Proof of Concept - Pipeline

---

**1. Cattura Passiva**  
OpenDrop monitora BLE/AWDL



**2. Estrazione Hash**  
Parsing VR in tempo reale



**3. Inversione**  
RainbowPhones lookup



**Requisiti: laptop + scheda Wi-Fi monitor + codice open-source su GitHub**



# 06 AirCollect: Risultati sperimentali

## Risultati Sperimentali (WiSec '21)

**99,8%**

Tasso successo inversione

**1,2<sub>ms</sub>**

Tempo medio per hash

**820**

Hash/sec throughput

### Test Sender Leakage

10 iPhone nelle vicinanze → **tutti i numeri recuperati in 12 secondi** .

Collo di bottiglia: trasmissione BLE, non inversione.

### Test Receiver Leakage

25 smartphone, pacchetto spoofato → **22/25 dispositivi hanno risposto** . I 3 non in modalità "Ricezione non attiva".

# 06 AirCollect: Implicazioni pratiche

---

## 1. Sorveglianza di Massa

Database di centinaia di numeri raccolti in pochi minuti in luoghi affollati.



## 2. Identificazione Retroattiva

Il *Log File Leakage* permette di identificare i target giorni dopo.



## 3. Profilazione Persistente

Tracciamento degli spostamenti fisici combinando gli hash.



## 4. Invisibilità Totale

Nessun avviso o notifica sul dispositivo della vittima.



# 07 Verifica Sperimentale della Vulnerabilità

---

## Configurazione Honeypot

**1** **Mac Attaccante**  
Configurato come ricevitore finto per intercettare traffico



**2** **iPhone Vittima**  
AirDrop in modalità "Tutti"



**3** **Intercettazione**  
Analisi del traffico durante autenticazione





# 07 Esperimento 1: Intercettazione con HoneyPot



## Procedura

- 1 Avvio Mac in modalità ricezione:  
`opendrop -d receive`
- 2 Configurazione iPhone vittima: AirDrop su modalità "Tutti"
- 3 Monitoraggio log HTTP sul server HTTPS locale di OpenDrop

## >\_ Log HTTP Intercettati

### POST /Discover

Dispositivo vittima annuncia presenza e interroga ricevitore

### POST /Ask

Invio payload con **hash dei contatti** senza validazione identità ricevitore

### POST /Upload

Tentativo di trasferimento file



## Risultato Chiave

Il protocollo procede nella negoziazione e **trasmette dati identificativi prima di stabilire un canale autenticato**, confermando il difetto progettuale fondamentale di AirDrop.

# 07 Leak Secondario: SenderPushToken

---



## Scoperta Critica

Dagli header della richiesta **POST /Upload**, il dispositivo vittima trasmette in chiaro il campo:

### **SenderPushToken:**

C5EA7D63CED8F2E97\*\*\*\*\*128E4C5

Questo token è un **identificatore univoco del dispositivo** utilizzato dal servizio Apple Push Notification (APNs).

Tracciamento Persistente



Superficie di Attacco Ampliata



Non Documentato



# 07 Esperimento 2: Spoofing URL



## Procedura

Invio URL arbitrario al dispositivo ricevente tramite comando:

```
opendrop send -r 0 -f https://owlink.org --url
```



## Risultato

L'azione si è completata con **successo** : il link è stato aperto sul dispositivo target.



## Vettore di Attacco

### Classificazione STRIDE

**Spoofing e Tampering** : l'attaccante può forzare l'apertura di URL arbitrari su dispositivi nelle vicinanze.

### Senza Consenso

La vittima **non ha accettato esplicitamente** un file: l'apertura è forzata dal protocollo.

### Contenuti Malevoli

Possibili payload: **pagine di phishing** , malware, exploit browser.

# 07

## Esperimento 3: Evoluzione dell'Attacco in iOS 16/17



### Procedura

1

#### Avvio Honeypot

Mac in modalità `opendrop -d receive`

2

#### Osservazione Passiva

Monitoraggio comportamento server prima e dopo interazione utente

3

#### Interazione Attiva

Vittima seleziona ricevitore finto sullo schermo



### Risultati Chiave

#### iOS 16/17: Comportamento Cambiato

Il broadcast automatico degli hash non si verifica più in assenza di interazione utente.

#### Vulnerabilità Persistente

Al momento dell'interazione, la sequenza `/Discover` → `/Ask` → `/Upload` si ripete identica .



### Conclusione Critica

Apple ha modificato il **comportamento** di AirDrop, ma la **radice crittografica del difetto** — l'uso di hash SHA-256 senza salt su numeri di telefono a bassa entropia — **rimane invariata** .

# 08 PrivateDrop: La Soluzione

## Private Set Intersection (PSI)

Protocollo crittografico che permette a due parti di confrontare insiemi privati, rivelando **solo l'intersezione**. Usato da Google Chrome per verificare password compromesse senza inviarle ai server.

L'obiettivo è la **verifica reciproca** prima di inviare dati sensibili.

- Fase 1:** Il Mittente invia la rubrica cifrata, il Destinatario verifica se i propri identificatori sono presenti. (Il destinatario scopre se il mittente lo conosce).
- Fase 2:** Ruoli invertiti. Il Destinatario invia la rubrica cifrata, il Mittente si cerca all'interno. (Il mittente scopre se il destinatario lo conosce).





# 08 PrivateDrop: Crittografia

## La "Cassaforte a Doppia Serratura" (Curve Ellittiche)

Nessuno dei due dispositivi vede i dati in chiaro: il confronto avviene interamente su valori crittografici, garantendo la privacy assoluta.



### 1. Offuscamento Iniziale

Il Destinatario trasforma i propri ID in punti su curva ellittica, li maschera con una chiave casuale e li invia.



### 2. Doppia Cifratura

Il Mittente riceve i dati, applica la propria chiave segreta (il "secondo lucchetto") e li rimanda indietro.



### 3. Sblocco Parziale:

Il Destinatario rimuove il proprio offuscamento iniziale. Ora l'ID è cifrato *solo* con la chiave del Mittente.



### 4. Confronto Sicuro

Il Mittente invia la sua rubrica, cifrata con la propria chiave. Il Destinatario confronta i valori crittografati per trovare le corrispondenze.

# 08 PrivateDrop: La Soluzione

## Ottimizzazioni di Performance

- ⚡ **Precomputazione:** operazioni pesanti in carica/inattivo
- ↗ **Riduzione messaggi:** da 4 a 2 round di comunicazione
- 📦 **Padding:** 10.000 voci rubrica, 10 ID

## Valutazione Performance

| Scenario              | AirDrop | PrivateDrop |
|-----------------------|---------|-------------|
| Senza precomputazione | ~200 ms | ~800 ms     |
| Con precomputazione   | ~200 ms | ~400 ms     |
| Rubrica > 10.000 voci | ~200 ms | < 1.000 ms  |

Overhead impercettibile per l'utente. Collo di bottiglia: latenza rete, non crittografia (<50 ms)



**Protezione input malevoli:** solo identificatori Apple-verificati possono entrare nel protocollo PSI

# 09 Risposta di Apple e Implicazioni

Nov  
2022

**iOS 16.1.1**  
**Limite 10 minuti "Tutti"**  
Inizialmente **solo Cina**  
(pressioni governo dopo  
proteste anti-Xi)

Dic  
2022

**iOS 16.2**  
**Estensione globale**  
Giustificata come misura  
"anti-spam". **Non risolve la  
vulnerabilità crittografica**

Gen  
2024

**Caso reale - Autorità cinesi**  
Hanno **identificato**  
manifestanti tramite Log File  
Leakage + rainbow table su  
log sysdiagnose

## Responsible Disclosure

Heinrich et al. hanno comunicato le vulnerabilità ad Apple Product Security **prima** di USENIX Security 2021. Codice open-source disponibile su GitHub.

# 09 Risposta di Apple e Implicazioni

## Il Problema Persiste

Apple **non ha integrato PrivateDrop** nelle versioni pubbliche. AirDrop continua a usare hash SHA-256 senza salt. La patch crittografica esiste, è pubblica, validata — ma non distribuita.

### Analisi Critica

- ✓ Correzione tecnica disponibile e validata sperimentalmente
- ✗ Scelta aziendale di non implementare la soluzione crittografica
- ! Misure adottate sono **politiche/UX** (limite tempo), non tecniche
- 🌐 Milioni di dispositivi rimangono esposti per **scelta**, non impossibilità



# 10 Conclusioni

---

01

## Scelta Progettuale, Non Bug:

Trattare SHA-256 su dati a bassa entropia (numeri di telefono) come protezione dell'identità è un errore di design alla radice, vulnerabile alla precomputazione.



02

## Verifica Sperimentale

I test empirici su iOS 16/17 confermano che l'attacco si è evoluto in *Honeypot Attivo*. La vulnerabilità crittografica su /Ask e il leak del SenderPushToken sono ancora presenti.



03

## La Soluzione Ignorata

*PrivateDrop* (basato su PSI) risolverebbe il problema alla radice con latenze di <1s, ma non è stato implementato.



Domande



Corso di Laurea Magistrale in Informatica



Programmazione Sicura 2025-2026

**GRAZIE PER  
L'ATTENZIONE**

<https://github.com/cost-02/ProgettoPS>



Costantino Paciello  
Vittorio Guida